

**LICENCIATURA EN
INFORMÁTICA INGENIERÍA
EN INFORMÁTICA**

Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°6**

Pipelining (Segmentación)

Rodriguez del Busto, Sofia

1) Rendimiento de Ciclo-Único versus Pipeline

- a) Complete la tabla y compare el tiempo promedio entre las siguientes instrucciones en una implementación de ciclo-único (todas las instrucciones toman un ciclo de reloj), a una implementación con pipeline; sabiendo que los tiempos de operación de las principales funciones son: 300 ps para acceso a memoria, 250 ps para operación con la ALU y 150 ps para lectura de registro y 200 ps para escritura de registro.

lw \$t1, 50(\$t0)
add \$s0, \$t2, \$t3
sw \$t3; 0(\$t0)

Tipo de Instrucción	Buscar instrucción (Fetch)	Lectura de registro	Operación ALU	Acceso a datos	Escritura de registro	Tiempo Total
Cargar palabra	300 ps	150 ps	250 ps	300 ps	200 ps	1200 ps
Guardar palabra	300 ps	150 ps	250 ps	300 ps		1000 ps
Suma	300 ps	150 ps	250 ps		200 ps	900 ps

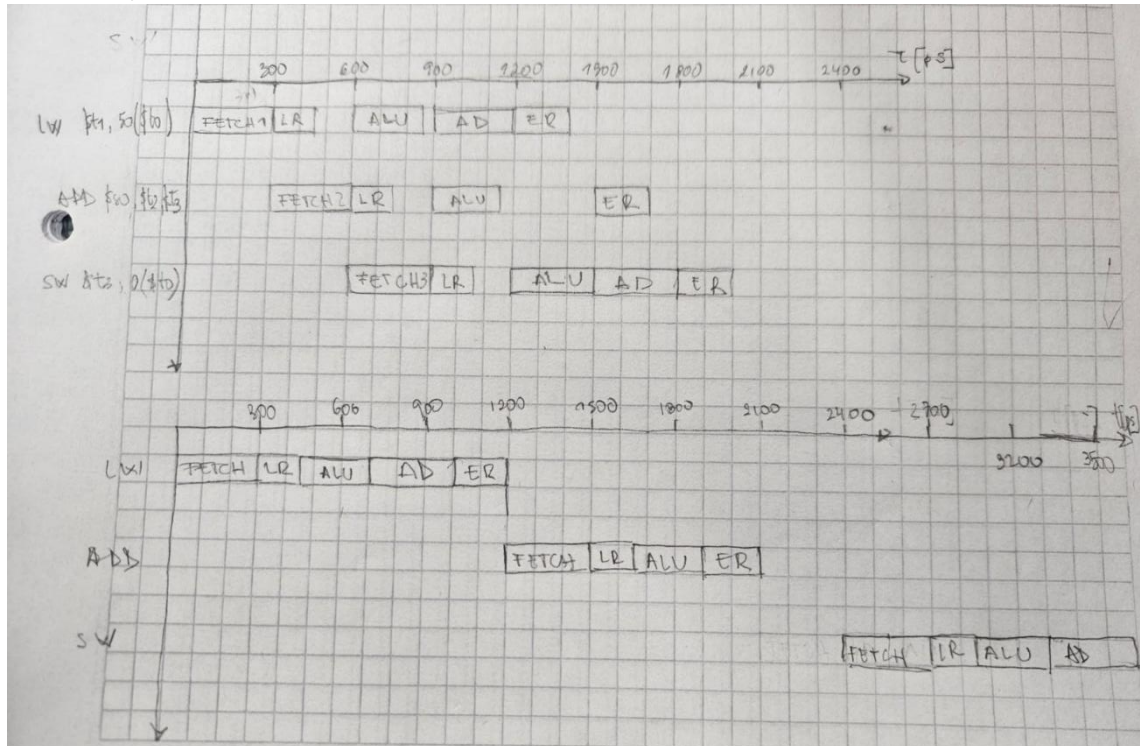
- b) Realice dicha comparación en un gráfico en función del tiempo.



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°6**

Pipelining (Segmentación)

Rodriguez del Busto, Sofia



En una implementación de ciclo único, todas las instrucciones se ejecutan en un único ciclo de reloj, sin importar su complejidad. Esto significa que el período de reloj debe ser lo suficientemente largo como para permitir que la instrucción más lenta complete todas sus etapas dentro de ese único ciclo. En este caso, la instrucción más lenta es lw (load word), cuyo tiempo total es 1200 s.

El tiempo de ejecución total será 3 instrucciones * 1200 ps/instrucción = 3600 ps. Y el tiempo promedio por instrucción es 1200 ps.

En una implementación pipeline, las instrucciones se dividen en etapas que se ejecutan de manera paralela. Cada etapa se completa en un ciclo de reloj distinto, y el período de reloj está determinado por la etapa más lenta. En este caso el acceso a memoria: 300 ps. Vemos que el tiempo de ejecución total es 2100 ps. Y el tiempo promedio por instrucción es 2100 ps / 3 = 700 ps.

c) ¿Hay algún Stall de pipeline en las líneas de código? Justifique su respuesta.

En esta secuencia de instrucciones no se producen stalls en el pipeline ya que no existen dependencias de datos entre ellas. La instrucción lw escribe en \$t1, pero la siguiente instrucción add no utiliza ese registro. Add escribe en \$s0, mientras que sw opera con \$t3, por lo que tampoco hay dependencia add y sw. Tampoco hay hazards de control ya que no hay instrucciones de salto. Finalmente, los structural hazards no será considerado ya que asumo la arquitectura típica con memorias separadas para instrucciones y datos.

2) Procesos con etapas, responda a las siguientes preguntas:

a) Si un proceso tiene 5 etapas de 1 hora de duración cada una y permite utilizar el concepto de pipelining. ¿Cuántas horas tomará realizar el proceso 4 veces?



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°6**

Pipelining (Segmentación)

Rodríguez del Busto, Sofia

En un pipeline, mientras una instrucción avanza a la etapa siguiente, la instrucción siguiente puede comenzar en la primera etapa. Por eso, la primera ejecución necesita tantos ciclos como etapas tiene el pipeline, porque debe atravesarlas todas. Luego, cada nueva ejecución termina exactamente un ciclo después que la anterior, ya que el pipeline está lleno y produce un resultado por ciclo. De este modo, el tiempo total es la suma de los ciclos necesarios para completar la primera instrucción más un ciclo adicional por cada una de las ejecuciones restantes.

La cantidad de ciclos entonces será: cantidad de etapas del pipeline + cantidad de ejecuciones – 1. En este caso, $5 + 4 - 1 = 8$. Como cada una dura una hora, la cantidad de tiempo que tomará realizar el proceso es 8 horas.

- b) Si un proceso tiene 5 etapas de 1 hora de duración cada una y permite utilizar el concepto de pipelining. ¿Cuántas horas tomará realizar el proceso 5 veces?

Por lo mencionado en el inciso a, sabemos que la cantidad de ciclos de este caso será: cantidad de etapas del pipeline + cantidad de ejecuciones – 1. En este caso, $5 + 5 - 1 = 9$. Como cada una dura una hora, la cantidad de tiempo que tomará realizar el proceso es 9 horas.

- c) ¿Cuánto es la mejora de un proceso de 4 etapas de 1 hora cada una si se lo aplica en un pipelining ideal para hacer 7 veces el mismo proceso?

Sin pipelining, la ejecución se da de forma secuencial. Como el proceso cuenta con 4 etapas de 1 hora, la duración del mismo es de 4 horas. Si ejecutamos 7 veces el proceso de manera secuencial, el tiempo total de ejecución será 7×4 horas = 28 horas.

Con pipelining, como calculamos anteriormente, la cantidad de ciclos es: cantidad de etapas del pipeline + cantidad de ejecuciones – 1. En este caso, $4 + 7 - 1 = 10$. Como cada una dura una hora, la cantidad de tiempo que tomará realizar el proceso es 10 horas.

La mejora porcentual usando pipelining es: $((28 - 10)/28) \times 100 = 64,28\%$.

- d) ¿Pipelining permite reducir el tiempo de ejecución de una única instrucción o reduce el tiempo total de un conjunto de instrucciones?

No, la latencia de una instrucción individual no se reduce con pipelining. Se reduce el tiempo total/medio por instrucción en un conjunto grande de instrucciones, porque las etapas se ejecutan en paralelo y se completa una instrucción por ciclo una vez que está lleno..

- e) ¿Cuáles podrían ser los posibles inconvenientes de tener un pipeline con demasiadas etapas?

Un pipeline con demasiadas etapas puede generar varios inconvenientes. En primer lugar, cada etapa adicional requiere registros intermedios, lo que introduce overhead y hace que el control sea más complejo. También aumentan los riesgos de datos y la complejidad del forwarding, lo que puede requerir más hardware o provocar más stalls. Finalmente, dividir demasiado las etapas dificulta lograr que todas tengan la misma duración. Si una queda más lenta que el resto, termina determinando el ciclo de reloj y limita el rendimiento



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°6**

Pipelining (Segmentación)

Rodriguez del Busto, Sofia

total.

- f) ¿Qué sucede cuando el forwarding no es suficiente para resolver un riesgo de datos?

Cuando forwarding/bypassing no puede entregar el operando a tiempo, por ejemplo: load-use hazard, porque un lw obtiene el dato sólo después de la etapa MEM, el hardware detecta el hazard e inserta una o más burbujas (stalls) en el pipeline hasta que el dato esté disponible. Una alternativa para mitigar el costo en estos casos es el reordenamiento de instrucciones por el compilador (scheduling).

- g) ¿Los riesgos de control ocurren cuando el pipeline debe pararse porque un paso debe esperar para que otro se complete?

Los riesgos de control no ocurren porque una etapa del pipeline deba esperar el resultado de otra, sino porque el procesador no sabe con seguridad cuál será la próxima instrucción a ejecutar cuando aparece un salto o branch. Mientras la condición del salto aún no se ha evaluado, el pipeline puede estar cargando instrucciones que quizás deban descartarse, lo que obliga a detener o vaciar parte del pipeline hasta conocer el camino correcto. Los riesgos de control surgen por la incertidumbre sobre el flujo de ejecución.

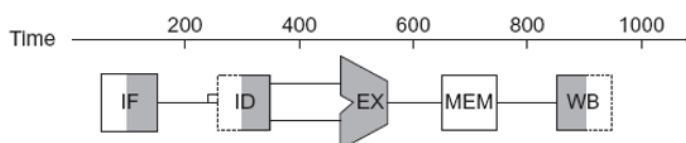
- 3) Represente en qué etapas del pipeline se puede aplicar forwarding para el siguiente código de instrucciones:

lw \$t0, 50(\$t1)

add \$t1, \$t0, \$t2

sub \$t2, \$t1, \$t3

Siendo las etapas de cada instrucción:



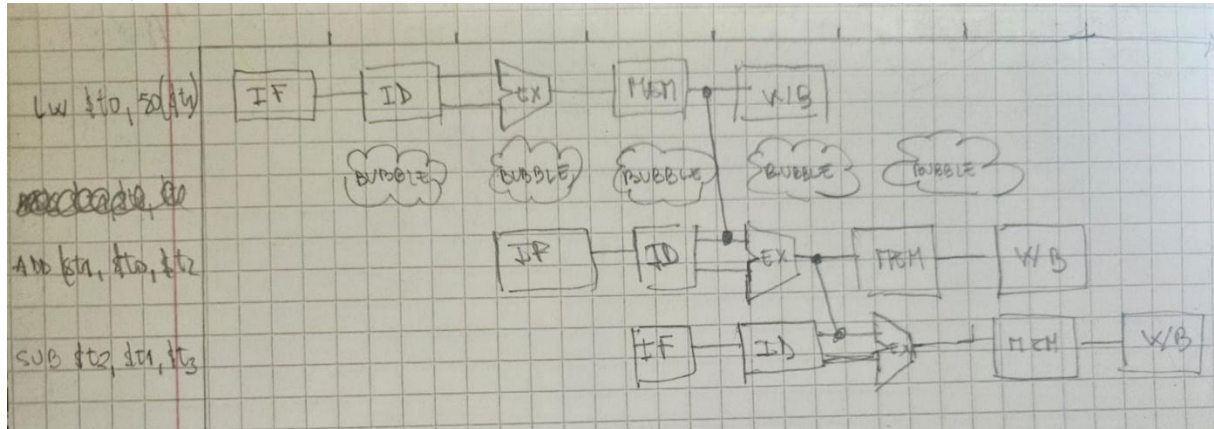
En estas instrucciones hay dos dependencias que pueden resolverse con forwarding. La primera es entre lw y add porque add necesita el valor cargado en \$t0. Ese dato recién está disponible al final de la etapa MEM del lw, por lo que el forwarding es desde MEM hacia EX. Esto requiere insertar un ciclo de burbuja ya que el dato no llega a tiempo para la EX del add. La segunda dependencia es entre add y sub ya que sub usa el valor que add produce en su ALU. En este caso, el resultado está disponible al final de EX del add, por lo que puede forwardearse directamente hacia la EX del sub sin necesidad de stalls.



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°6**

Pipelining (Segmentación)

Rodriguez del Busto, Sofia



- 4) Reordenar las instrucciones o agregar otras para el siguiente segmento de código para evitar paradas en el pipeline.

lw \$t1, 0(\$t0)
sub \$t2, \$t1, \$t3 – se produce load-use hazard con lw anterior
lw \$t2, 4(\$t0)
add \$t3, \$t1, \$t2 – se produce load-use hazard con lw anterior
sw \$t3, 12(\$t0) Hay dependencia pero con forwarding de EX de add a MEM de sw se puede solucionar
sw \$t2, 24(\$t0) – Depende del load anterior
lw \$t2, 16(\$t0) – load independiente no se utiliza ese valor para operar
lw \$t4, 8(\$t0) – Puedo cargar t4 en cualquier momento siempre y cuando sea antes del add siguiente pues no se modifica en otra instruccion
add \$t5, \$t1, \$t4
sw \$t5, 16(\$t0) Hay dependencia pero con forwarding de EX de add a MEM de sw se puede solucionar

Reordenado

lw \$t1, 0(\$t0) – Carga t1
lw \$t2, 16(\$t0) – Cubre load-use con sub
sub \$t2, \$t1, \$t3 – Usa t1, ya no produce stall por load-use hazard. Sobreescribe \$t2
lw \$t2, 4(\$t0) – Carga nuevo t2
lw \$t4, 8(\$t0) – Independiente cubre load-use con add
add \$t3, \$t1, \$t2 – Usa t1 y t2, ya no produce stall por load-use hazard
sw \$t3, 12(\$t0) – Depende de add anterior pero se soluciona con forwarding EX - MEM
sw \$t2, 24(\$t0) – Depende de lw \$t2 anterior para cuando llega a escribir en memoria el lw esta completo
add \$t5, \$t1, \$t4 – Usa t1 y t4
sw \$t5, 16(\$t0) – Depende de add anterior pero se soluciona con forwarding EX – MEM

- 5) Determine si existe algún tipo de riesgo y, en caso de existir, mencione de que tipo



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS II
TRABAJO PRÁCTICO N°6**

Pipelining (Segmentación)

Rodriguez del Busto, Sofia

a)

add \$s0, \$t0, \$t1

sub \$s0, \$t0, \$t2

Ambas instrucciones escriben en el mismo registro \$s0. En un pipeline in-order, como la MIPS de 5 etapas que estudiamos, no provoca problema funcional porque las escrituras llegan en orden. Sin embargo, es una dependencia que en diseños con out-of-order, donde se ejecutan las instrucciones tan pronto como sus datos estén listos, no necesariamente en el orden original del programa.

b)

add \$t3, \$t0, \$t1

add \$s0, \$t3, \$t2

La segunda instrucción lee \$t3, que es producido por la primera instrucción (add → t3) por lo que existe un riesgo de datos ya que la segunda instrucción necesita de datos que se producen en la primera. Esto se puede resolver con EX→EX forwarding (el resultado del primer add se envía a la EX de la siguiente) y por tanto no necesita stalls.